Future-Proof Your Treasury: Post-Quantum Accounts for Kohaku Wallet

Simon Masson

ZKNOX

April 1st, 2026 EthCC[9] Cannes, France

ZKNOX team

Nicolas Bacca
20^+ years experience (10^+y web3)
Security and hardware specialist
Prev. Ledger cofounder/CTO

Renaud Dubois
20^+ years experience (3^+y web3)
Cryptographer
Prev. Ledger, Thales

Simon Masson
8^+ years experience (4^+y web3)
Cryptographer
Prev. Heliax, Thales

Expertise and innovation to every challenge on the whole security chain:

- user end (secure enclaves, hardware wallets),
- back end (TEE, HSMs),
- on-chain (smart contracts).

Execution layer is protected from quantum computer now!

zknox.com

Kohaku

Wallet focusing on privacy and security:

- Overall presentation by N. Consigny, Monroe Stage (now)
- Trustless manifesto by S. Kux, Taylor Stage (after me)
- Privacy integration in hardware wallets by R. Dubois, Taylor Stage (yesterday)
- Account abstraction targeted soon, enabling integration of this work!

github.com/ethereum/kohaku

Why post-quantum?

Security of Ethereum relies on **digital signatures**:

- ECDSA: Elliptic Curve Digital Signature Algorithm:
  Small signatures: 65B.
  Fast verification, secure, and trusted.
  A quantum computer breaks the security!

Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits

Madelyn Cain[1,*,†], Qian Xu[1,2,*,‡], Robbie King[1], Lewis R. B. Picard[1], Harry Levine[1,3], Manuel Endres[1,2], John Preskill[1,2], Hsin-Yuan Huang[1,2], Dolev Bluvstein[1,2,§]
[1] *Oratomic, Pasadena, California 91125, USA*
[2] *California Institute of Technology, Pasadena, California 91125, USA*
[3] *Department of Physics, University of California, Berkeley, California 94720, USA*
[†] *mcain@oratomic.com*, [‡] *qxu@oratomic.com*, [§] *dbluvstein@oratomic.com*
[*] *These authors contributed equally*
(Dated: March 31, 2026)

Why post-quantum?

Security of Ethereum relies on **digital signatures**:

- ECDSA: Elliptic Curve Digital Signature Algorithm:
  Small signatures: 65B.
  Fast verification, secure, and trusted.
  A quantum computer breaks the security!

Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits

Madelyn Cain[1,*,†], Qian Xu[1,2,*,‡], Robbie King[1], Lewis R. B. Picard[1], Harry Levine[1,3],
Manuel Endres[1,2], John Preskill[1,2], Hsin-Yuan Huang[1,2], Dolev Bluvstein[1,2,§]
[1] *Oratomic, Pasadena, California 91125, USA*
[2] *California Institute of Technology, Pasadena, California 91125, USA*
[3] *Department of Physics, University of California, Berkeley, California 94720, USA*
[†] *mcain@oratomic.com*, [‡] *qxu@oratomic.com*, [§] *dbluvstein@oratomic.com*
[*] *These authors contributed equally*
(Dated: March 31, 2026)

- Alternative signatures resist against quantum algorithms:
  Public key and signature much larger (thousands of bytes).
  2016: NIST starts a standardization process.
  2026: Three candidates for post-quantum secure signature.
  Many institutions already integrate PQ security.
  Ethereum announced a PQ roadmap
  (Consensus, Data and Execution Layers, see strawmap.org).

Post-quantum signature candidates

Standard	FN-DSA	ML-DSA	SLH-DSA
Name	Falcon	Dilithium	Sphincs
FIPS	203	204	205
Family	Lattice	Lattice	Hash
Signature	666B	2420B	7856B
Public key	897B	1312B	32B
Gas cost verification without precompile	3.8M	8.0M	–
Hardware implementation			
Aggregation			
TSS			
PK recovery			

Post-quantum signature candidates

Standard	FN-DSA	ML-DSA	SLH-DSA
Name	Falcon	Dilithium	Sphincs
FIPS	203	204	205
Family	Lattice	Lattice	Hash
Signature	666B	2420B	7856B
Public key	897B	1312B	32B
Gas cost verification without precompile	3.8M	8.0M	–
Hardware implementation	more than 40kB	yes, see later in this talk	–
Aggregation	Possible	no	no
TSS	naive	yes	no
PK recovery	yes	no	no

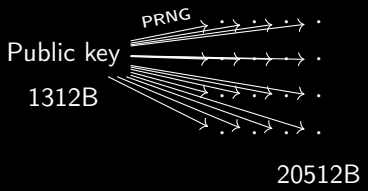
EIPs for lattice-based signatures

EIP-8051: **MLDSA**

Two precompiles:

- ML-DSA-NIST: with SHAKE256,
- ML-DSA-ETH: with KECCAK256.

PubKey expanded during verification into $(\mathbb{F}_{q^{256}}^{4\times 4}, \mathit{bytes}, \mathbb{F}_{q^{256}}^{4})$:



Verifier (after expansion):

- **9** SHA256 (or **5** Keccak256),
- **8** NTTs (256 length polynomials).

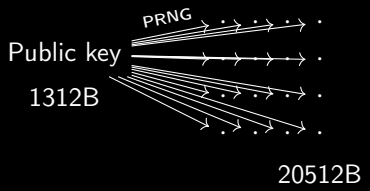
EIPs for lattice-based signatures

EIP-8051: **MLDSA**

Two precompiles:

- ML-DSA-NIST: with SHAKE256,
- ML-DSA-ETH: with KECCAK256.

PubKey expanded during verification into $(\mathbb{F}_{q^{256}}^{4\times 4}, \mathit{bytes}, \mathbb{F}_{q^{256}}^{4})$:



Verifier (after expansion):

- **9** SHA256 (or **5** Keccak256),
- **8** NTTs (256 length polynomials).

EIP-8052: **FNDSA**

Two precompiles (hash and sign):

- HashToPoint (NIST or variants),
- Core verification (NTTs).

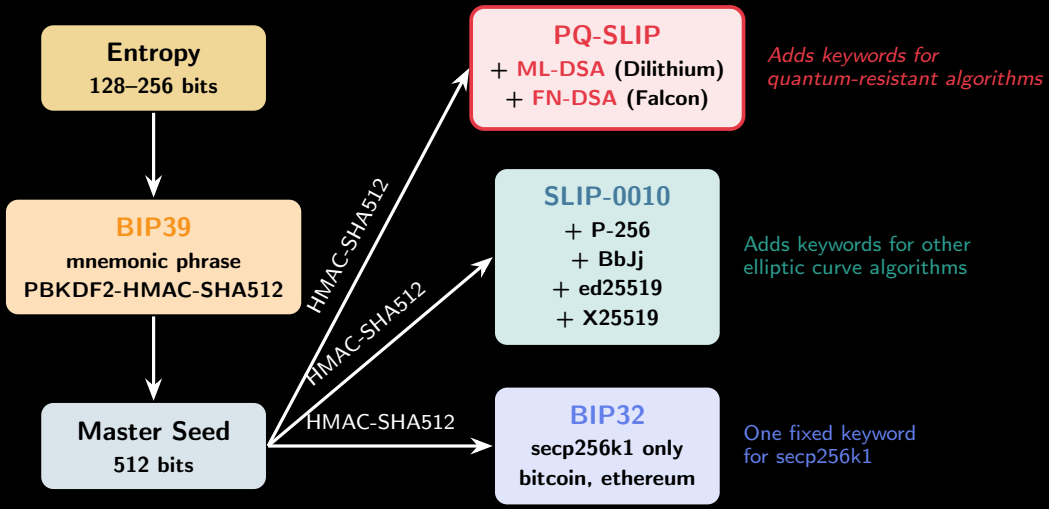
PubKey variation:

- Move to the NTT domain
- 896B $\rightarrow$ 1024B
- Remove one NTT for verifier

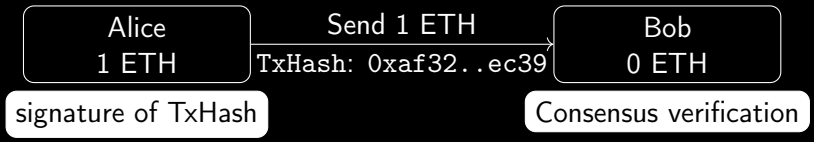
Verifier (in this formate):

- 35 SHA256 (or Keccak, Poseidon),
- 2 NTTs (512 length polynomials).

PQ-SLIP: how to handle PQ keys



Account Abstraction



- Execution layer: ECDSA signature (not resistant to quantum attacks)

Account Abstraction

- Execution layer: ECDSA signature (not resistant to quantum attacks)

Account Abstraction

- Execution layer: ECDSA signature (not resistant to quantum attacks)
- Account abstraction (4337): enables custom contract for transactions.

Account Abstraction

- Execution layer: ECDSA signature (not resistant to quantum attacks)
- Account abstraction (4337): enables custom contract for transactions.
- This work: 4337 contract with quantum-secure signature verification.

Hybrid verification

What is hybrid verification?

- Verify ECDSA signature (secure and trusted),
- Verify PQ signature (secure against a quantum computer).

$\Longrightarrow$ Nowadays trust and future security.

- Pre-quantum:

  ECDSA-k1 ECDSA-r1

- Post-quantum:
  - MLDSA NIST
  - MLDSA
  - FALCON NIST
  - FALCON

Demonstration: what do we show here?

Step 1. Create Account

- Requires an EOA that is already funded,
- Derive pre-quantum and post-quantum seed,
- Pre-quantum key generation: $(\mathsf{sk}_1, \mathsf{pk}_1)$,
  Post-quantum key generation: $(\mathsf{sk}_2, \mathsf{pk}_2)$,
- *The account is created* as a 4337 smart contract storing pk_1 and pk_2.

Demonstration: what do we show here?

Step 1. Create Account

- Requires an EOA that is already funded,
- Derive pre-quantum and post-quantum seed,
- Pre-quantum key generation: $(\mathsf{sk}_1, \mathsf{pk}_1)$,
  Post-quantum key generation: $(\mathsf{sk}_2, \mathsf{pk}_2)$,
- *The account is created* as a 4337 smart contract storing pk_1 and pk_2.

Step 2. Send Transaction

- Does not require an EOA anymore,
- The freshly created account needs to be funded,
- Compute the `UserOpHash` to be signed,
- Sign it with a pre-quantum scheme using sk_1,
  Sign it with a post-quantum scheme using sk_2,
- *A transaction is spent* from the 4337 account if:
  - the pre-quantum signature is valid w.r.t pk_1 stored in the contract,
  - the post-quantum signature is valid w.r.t pk_2 stored in the contract,

Demonstration: what do we show here?

Step 1. Create Account

- Requires an EOA that is already funded,
- Derive pre-quantum and post-quantum seed,
- Pre-quantum key generation: (sk_1, pk_1),
  Post-quantum key generation: (sk_2, pk_2),
- *The account is created* as a 4337 smart contract storing pk_1 and pk_2.

Step 2. Send Transaction

- Does not require an EOA anymore,
- The freshly created account needs to be funded,
- Compute the `UserOpHash` to be signed,
- Sign it with a pre-quantum scheme using sk_1,
  Sign it with a post-quantum scheme using sk_2,
- *A transaction is spent* from the 4337 account if:
  - the pre-quantum signature is valid w.r.t pk_1 stored in the contract,
  - the post-quantum signature is valid w.r.t pk_2 stored in the contract,

pq4337.netlify.app

Conclusion and perspectives

Smart accounts with post-quantum security are already available!

- ☑ Experimentation with MLDSA and FALCON (at ETH price of today):

Signatures	Create Account	Send Transaction
ECDSA + MLDSA	5.7M gas ($5.50)	8.2M gas ($7.90)
ECDSA + MLDSA ⟠		5.0M gas ($4.80)
ECDSA + FALCON	1.0M gas ($1.00)	3.8M gas ($3.60)
ECDSA + FALCON ⟠		1.6M gas ($1.50)

- ☐ Production: MLDSA-65 for higher security level
- ☐ Production: audit of the code
- ☑ Hardware: MLDSA, first implementation fully running in secure element
- ☐ Hardware: FALCON reaches the 40kB limit.

Our solution is expensive. We need precompiles!

Future of PQ accounts

- ☐ Delegation (EIP-7702) for spending from the EOA,
- ⏲ Deactivation of EOA (EIP-7851),
- ⏲ Frame transaction (EIP-8141) integration.
- ⏲ Specific precompiles:
  - EIP-8051 (MLDSA): around 4.5k gas,
  - EIP-8052 (FALCON): around 3k gas.
- ⏲ Generic precompile (NVMPy): vectorized maths arithmetic.
  Ongoing work with Vitalik: estimated full falcon around 50k gas
  Remaining cost: data call/store/copy ⚠

On-chain data gas cost (lower bound estimation)

Account Mode	Create Account	Gas	Send Transaction	Gas
ecrecover → falcon ↗↘	create2	32000	extcodecopy	2600
			call data sig	$65B \times 16 = 1040$
	call data pk	$896B \times 16 = 14336$	call data sig	$666B \times 16 = 10656$
	store pk	$896B \times 200 = 179200$	copy pk	$896B \times 100 \div 32 = 2800$
	total	226k ($0.22)	total	17k ($0.02)
ecrecover → falconrecover →	create2	32000		
			call data sig	$65B \times 16 = 1040$
			call data sig	$1291B \times 16 = 20656$
	total	32k ($0.03)	total	22k ($0.02)
ecrecover → mldsa ↗↘	create2	32000	extcodecopy	2600
			call data sig	$65B \times 16 = 1040$
	call data pk	$1312B \times 16 = 20992$	call data sig	$2420B \times 16 = 38720$
	store pk	$1312B \times 200 = 262400$	copy pk	$1312B \times 100 \div 32 = 4100$
	total	315k ($0.30)	total	46k ($0.04)

Thank you for your attention

pq4337.netlify.app

Create and use your quantum-secure account (on Sepolia)!